

Table Per Concrete Type (TPC) Inheritance in Entity Framework Core

In this article, I will discuss **Table Per Concrete Type (TPC) Inheritance**, how to implement it in Entity Framework Core (EF Core), and provide a step-by-step real-time example. Please read our previous two articles discussing [Table Per Type \(TPT\)](#) and [Table Per Hierarchy \(TPH\)](#) Inheritance in EF Core with Examples.

Table Per Concrete Type (TPC) Inheritance in Entity Framework Core

Table Per Concrete Type (TPC) is an inheritance mapping strategy where each concrete class in an inheritance hierarchy is mapped to its own database table. Unlike Table Per Hierarchy (TPH) and Table Per Type (TPT), TPC does not include a table for the abstract base class. Instead, all properties, including those inherited from the base class, are stored in the derived class's table. This approach eliminates the need for joins when querying derived types since all the data resides in a single table per concrete class.

Key Features of Table Per Concrete Type (TPC) Inheritance in EF Core:

- **Separate Tables for Each Concrete Class:** Each concrete class in the hierarchy is mapped to its own table.
- **Complete Entity Representation:** Each table fully represents a concrete entity, including inherited properties.
- **No Table for Abstract Base Class:** The base class is not represented by a separate table.
- **No Joins Required:** Queries against derived types do not require joins, potentially improving performance.
- **No Foreign Key Relationships:** Unlike TPT, there are no relationships or foreign keys between base and derived classes.
- **No Discriminator Column:** Unlike TPH, TPC does not require a discriminator column to identify the type of the entity.
- **No Null Columns:** Unlike Table Per Hierarchy (TPH), TPC avoids null columns by ensuring each table only contains columns relevant to its entity.

Note: TPC inheritance can lead to data duplication due to repeated base class properties across multiple tables. However, it offers performance benefits by eliminating the need for joins when querying specific concrete types.

How to Implement Table Per Concrete Type (TPC) Inheritance in Entity Framework Core

Implementing TPC inheritance in EF Core involves defining the base and derived classes, configuring the inheritance mapping using the Fluent API, and managing the database schema accordingly. So, we need to follow the below steps to Implement TPC Inheritance in EF Core:

- **Create the Base Class and Derived Classes:** Define the base class for shared properties and derived classes for specific properties.
- **Configure TPC Using Fluent API:** Use the Fluent API in the `OnModelCreating` method to specify that each concrete class should map to its own table using the `.ToTable()` method.
- **Use the `UseTpcMappingStrategy()` Method:** Call this method on the `ModelBuilder` to enable TPC mapping on the Base Entity.

Real-Time Example: Implementing TPC Inheritance in a Billing System

Consider a scenario where we are developing a Billing System that handles different types of invoices, such as Utility Bills, Product Purchases, and Subscription Services. Each invoice type shares common attributes (e.g., `InvoiceNumber`, `Amount`, `BillingDate`, etc.) but also has properties specific to that type of invoice. We will implement TPC inheritance to model this hierarchy in EF Core. Let's define the base and derived entities.

Base Entity: Invoice

Create a new class file named **Invoice.cs** within the Entities folder and add the following code. The Invoice class serves as the abstract base class for all invoice types.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    // Abstract Base Class representing a general Invoice
    public abstract class Invoice
    {
        public int InvoiceId { get; set; } // Primary Key
        public string InvoiceNumber { get; set; } // Unique Invoice Number
        [Column(TypeName = "decimal(18,2)")]
        public decimal Amount { get; set; } // Invoice Amount
        public DateTime BillingDate { get; set; } // Date of Billing
        public string CustomerName { get; set; } // Customer Name
        public string CustomerEmail { get; set; } // Customer Email
        public string BillingAddress { get; set; } // Customer's Billing
        Address
        public InvoiceStatus Status { get; set; } // Invoice Status (Paid,
        Pending, Overdue)
    }
    // Enum representing the status of the invoice
    public enum InvoiceStatus
    {
        Pending,
        Paid,
        Overdue
    }
}
```

Derived Entity: UtilityBill

Create a new class file named **UtilityBill.cs** within the Entities folder and add the following code. The UtilityBill class inherits from the Invoice and includes properties specific to utility bills.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    // Derived Class representing a Utility Bill
    public class UtilityBill : Invoice
    {
        public string UtilityType { get; set; } // Type of Utility (e.g.,
        Electricity, Gas, Water)
        public string MeterNumber { get; set; } // Meter Number for the
        utility
        [Column(TypeName = "decimal(18,2)")]
        public decimal UsageAmount { get; set; } // Quantity of Utility Used
        (e.g., kWh, gallons), not the Amount
        [Column(TypeName = "decimal(18,2)")]
        public decimal RatePerUnit { get; set; } //Rate Per Unit
    }
}
```

```

        public DateTime ServicePeriodStart { get; set; } // Start of Service
Period
        public DateTime ServicePeriodEnd { get; set; } // End of Service
Period
        public string UtilityProvider { get; set; } // Name of the utility
provider
        public DateTime DueDate { get; set; } // Due Date for the bill
    }
}

```

Derived Entity: ProductPurchase

Create a new class file named **ProductPurchase.cs** within the Entities folder and add the following code. The ProductPurchase class inherits from Invoice and includes properties specific to product purchases.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    // Derived Class representing a Product Purchase
    public class ProductPurchase : Invoice
    {
        public string ProductName { get; set; } // Name of the Product
        public int Quantity { get; set; } // Quantity Purchased
        [Column(TypeName = "decimal(18,2)")]
        public decimal UnitPrice { get; set; } // Price per Unit
        public string Vendor { get; set; } // Vendor from whom the product
was purchased
        [Column(TypeName = "decimal(18,2)")]
        public decimal ShippingCost { get; set; } // Shipping cost for the
product
        public string TrackingNumber { get; set; } // Shipment tracking
number
    }
}

```

Derived Entity: SubscriptionService

Create a new class file named **SubscriptionService.cs** within the Entities folder and add the following code. The SubscriptionService class inherits from Invoice and includes properties specific to subscription services.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    // Derived Class representing a Subscription Service
    public class SubscriptionService : Invoice
    {
        public string ServiceName { get; set; } // Name of the
Subscription Service
        public DateTime SubscriptionStart { get; set; } // Subscription Start
Date
    }
}

```

```

        public DateTime SubscriptionEnd { get; set; } // Subscription End
Date
        [Column(TypeName = "decimal(18,2)")]
        public decimal SubscriptionFee { get; set; } //Subscription Fee
        public string RenewalFrequency { get; set; } // Renewal Frequency
(e.g., Monthly, Annually)
        public bool AutoRenew { get; set; } // Indicates if the
subscription auto-renews
    }
}

```

Configuring the DbContext

Modify the EFCoreDbContext class as follows. This class represents the session with the database and includes the configuration necessary for EF Core to map entities to the database using TPC inheritance.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the connection string to the SQL Server database
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=InvoiceDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // Configures the model and mappings between entities and database
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configure the base class as abstract to prevent EF Core from creating a table
            modelBuilder.Entity<Invoice>().UseTpcMappingStrategy();
            // Configure TPC inheritance by mapping each concrete class to its base class
            modelBuilder.Entity<UtilityBill>();
            modelBuilder.Entity<ProductPurchase>();
            modelBuilder.Entity<SubscriptionService>();
        }
        // DbSet representing each table in the database
        public DbSet<Invoice> Invoices { get; set; } //No Table for this Property
        public DbSet<UtilityBill> UtilityBills { get; set; }
        public DbSet<ProductPurchase> ProductPurchases { get; set; }
        public DbSet<SubscriptionService> SubscriptionServices { get; set; }
    }
}

```

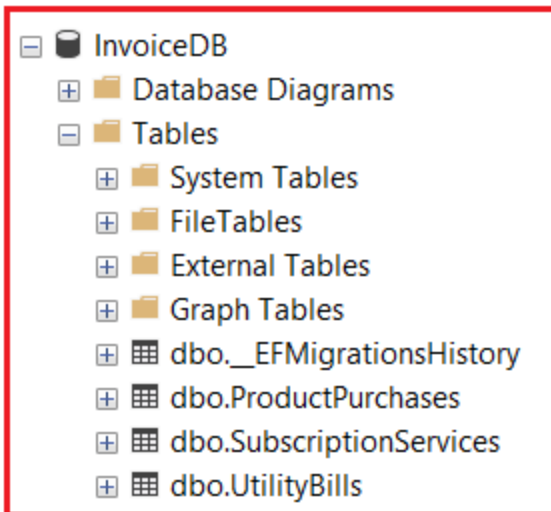
Note: Even though there is no table for Invoice due to TPC mapping, we include DbSet<Invoice> to enable polymorphic queries across all invoice types.

Adding Migration and Updating the Database

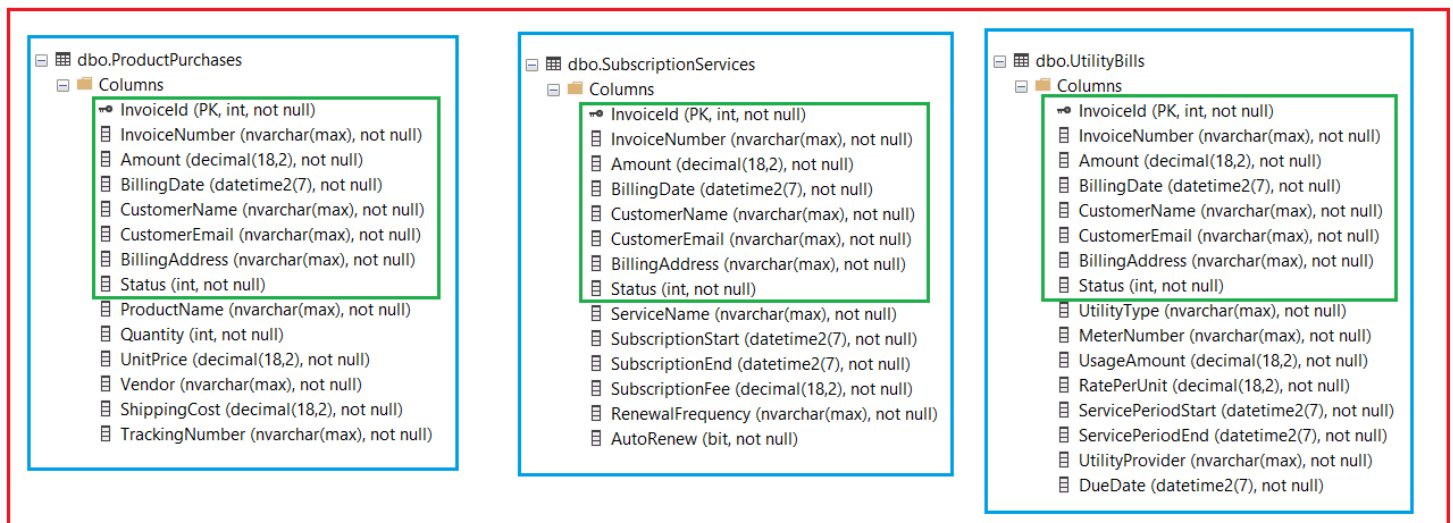
Open the Package Manager Console and execute the following commands to add a migration and update the database:

- **Add-Migration MIG1**
- **Update-Database**

After executing these commands, the database should be created with separate tables for each derived class (UtilityBills, ProductPurchases, and SubscriptionServices). As shown in the image below, there will be no table for the abstract base class Invoice.



If you verify the column structure of each table, you will see that the Base class properties are created as columns in each table, as shown in the image below. The InvoiceId also becomes the Primary key in each table.



Example of Insert Operations Using TPC Inheritance

Please modify the Program class as follows. The following example shows how to insert data into the UtilityBill, ProductPurchase, and SubscriptionService tables using the updated entities.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
```

```

{
    static void Main(string[] args)
    {
        using (var context = new EFCoreDbContext())
        {
            // Create and seed a Utility Bill
            var utilityBill = new UtilityBill
            {
                InvoiceNumber = "UB001",
                BillingDate = DateTime.Now,
                CustomerName = "Ravi Kumar",
                CustomerEmail = "ravi.kumar@example.com",
                BillingAddress = "123 Elm Street",
                Status = InvoiceStatus.Pending,
                UtilityType = "Electricity",
                MeterNumber = "MTR12345",
                UsageAmount = 250.75m,
                RatePerUnit = 0.40m, // 0.40 per unit of electricity
                ServicePeriodStart = DateTime.Now.AddMonths(-1),
                ServicePeriodEnd = DateTime.Now,
                UtilityProvider = "ElectricCo",
                DueDate = DateTime.Now.AddDays(15)
            };
            // Calculate the Amount for the utility bill
            utilityBill.Amount = utilityBill.UsageAmount *
utilityBill.RatePerUnit;
            // Create and seed a Product Purchase
            var productPurchase = new ProductPurchase
            {
                InvoiceNumber = "PP001",
                BillingDate = DateTime.Now,
                CustomerName = "Alice Johnson",
                CustomerEmail = "alice.johnson@example.com",
                BillingAddress = "456 Oak Avenue",
                Status = InvoiceStatus.Paid,
                ProductName = "Laptop",
                Quantity = 1,
                UnitPrice = 1500.00m,
                Vendor = "TechStore",
                ShippingCost = 25.00m,
                TrackingNumber = "TRACK123456789"
            };
            // Calculate the Amount for the product purchase
            productPurchase.Amount = (productPurchase.Quantity *
productPurchase.UnitPrice) + productPurchase.ShippingCost;
            // Create and seed a Subscription Service
            var subscriptionService = new SubscriptionService
            {
                InvoiceNumber = "SS001",
                Amount = 99.99m, //Amount equals to SubscriptionFee
                BillingDate = DateTime.Now,

```

```

        CustomerName = "Michael Scott",
        CustomerEmail = "michael.scott@example.com",
        BillingAddress = "789 Maple Street",
        Status = InvoiceStatus.Pending,
        ServiceName = "Netflix Subscription",
        SubscriptionStart = DateTime.Now,
        SubscriptionEnd = DateTime.Now.AddYears(1),
        SubscriptionFee = 99.99m,
        RenewalFrequency = "Annually",
        AutoRenew = true
    };
    // Add all invoices to the context
    context.UtilityBills.Add(utilityBill);
    context.ProductPurchases.Add(productPurchase);
    context.SubscriptionServices.Add(subscriptionService);
    // Save changes to the database
    int recordsAdded = context.SaveChanges();
    // Output the result
    Console.WriteLine($"{recordsAdded} records were saved to the
database.");
    }
}
}
}

```

Output: 3 records were saved to the database.

Explanation of Insert Operations:

- **Utility Bill:** The UtilityBill instance captures utility-specific data like UtilityType, MeterNumber, UsageAmount, and the billing period (ServicePeriodStart, ServicePeriodEnd). We calculate the Amount by multiplying UsageAmount by RatePerUnit.
- **Product Purchase:** The ProductPurchase instance contains product-specific data, including the product details (ProductName, UnitPrice, and Quantity), as well as the Vendor, ShippingCost, and TrackingNumber. We calculate the Amount by adding the product total (Quantity * UnitPrice) and ShippingCost.
- **Subscription Service:** The SubscriptionService instance captures subscription-related information such as ServiceName, SubscriptionFee, and whether the subscription is set to auto-renew (AutoRenew). Here, the Amount is equal to the SubscriptionFee.

Example of Read Operations Using TPC Inheritance in EF Core

Let us understand how to read and display data from the database for each of the invoice types. So, please modify the Program class as follows:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())

```

```

{
    // Retrieve and display all Utility Bills
    var utilityBills = context.UtilityBills.ToList();
    Console.WriteLine("\n--- List of Utility Bills ---");
    foreach (var bill in utilityBills)
    {
        Console.WriteLine($"Invoice: {bill.InvoiceNumber},
Amount: {bill.Amount}, Utility: {bill.UtilityType}, Usage: {bill.UsageAmount}
units, Rate: {bill.RatePerUnit}, Provider: {bill.UtilityProvider}");
    }
    // Retrieve and display all Product Purchases
    var productPurchases = context.ProductPurchases.ToList();
    Console.WriteLine("\n--- List of Product Purchases ---");
    foreach (var purchase in productPurchases)
    {
        Console.WriteLine($"Invoice: {purchase.InvoiceNumber},
Product: {purchase.ProductName}, Quantity: {purchase.Quantity}, Vendor:
{purchase.Vendor}, Shipping Cost: {purchase.ShippingCost}");
    }
    // Retrieve and display all Subscription Services
    var subscriptions = context.SubscriptionServices.ToList();
    Console.WriteLine("\n--- List of Subscription Services ---");
    foreach (var subscription in subscriptions)
    {
        Console.WriteLine($"Invoice:
{subscription.InvoiceNumber}, Service: {subscription.ServiceName}, Fee:
{subscription.SubscriptionFee}, Auto-Renew: {subscription.AutoRenew}");
    }
}
}
}
}

```

This example retrieves and lists all the data from each table (UtilityBills, ProductPurchases, and SubscriptionServices) and displays specific fields relevant to each invoice type.

- For Utility Bills, it displays data like UtilityType, UsageAmount, RatePerUnit, and UtilityProvider.
- For Product Purchases, it displays the Product Name, Quantity, Vendor, and shipping-related information.
- For Subscription Services, it displays the ServiceName, SubscriptionFee, and whether the service is set to auto-renew.

Now, run the application, and you should see the following output:

```

--- List of Utility Bills ---
Invoice: UB001, Amount: 100.30, Utility: Electricity, Usage: 250.75 units, Rate: 0.40, Provider: ElectricCo

--- List of Product Purchases ---
Invoice: PP001, Product: Laptop, Quantity: 1, Vendor: TechStore, Shipping Cost: 25.00

--- List of Subscription Services ---
Invoice: SS001, Service: Netflix Subscription, Fee: 99.99, Auto-Renew: True

```

Example of Update Operations Using TPC Inheritance in EF Core

Let us understand how we can update existing records in the database using TPC inheritance. For a better understanding, please modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // Update an existing Utility Bill
                var utilityBill = context.UtilityBills.FirstOrDefault(b =>
b.InvoiceNumber == "UB001");
                if (utilityBill != null)
                {
                    utilityBill.Amount = 110.50m; // Update the total amount
                    utilityBill.DueDate = DateTime.Now.AddDays(10); // Update
the due date

                    context.SaveChanges();
                    Console.WriteLine("Utility Bill updated.");
                }
                // Update an existing Product Purchase
                var productPurchase =
context.ProductPurchases.FirstOrDefault(p => p.InvoiceNumber == "PP001");
                if (productPurchase != null)
                {
                    productPurchase.Quantity = 2; // Increase the quantity
                    productPurchase.Amount = productPurchase.Quantity *
productPurchase.UnitPrice + productPurchase.ShippingCost;
                    context.SaveChanges();
                    Console.WriteLine("Product Purchase updated.");
                }
                // Update an existing Subscription Service
                var subscription =
context.SubscriptionServices.FirstOrDefault(s => s.InvoiceNumber == "SS001");
                if (subscription != null)
                {
                    subscription.AutoRenew = false; // Disable auto-renewal
                    context.SaveChanges();
                    Console.WriteLine("Subscription Service updated.");
                }
            }
        }
    }
}
```

Output:

Utility Bill updated.
Product Purchase updated.
Subscription Service updated.

Explanation of Update Operations:

- **Utility Bill:** The Amount and DueDate fields are updated for an existing utility bill.
- **Product Purchase:** The Quantity of the product is updated, and the total amount is recalculated based on the updated quantity.
- **Subscription Service:** The AutoRenew field is updated to disable automatic renewal.

Example of Delete Operations Using TPC Inheritance in EF Core

Finally, let us understand how we can delete existing records in the database using TPC inheritance. For a better understanding, please modify the Program class as follows. Utility Bill, Product Purchase, and Subscription Service records are deleted from their respective tables by finding the matching InvoiceNumber.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // Delete a Utility Bill
                var utilityBill = context.UtilityBills.FirstOrDefault(b =>
b.InvoiceNumber == "UB001");
                if (utilityBill != null)
                {
                    context.UtilityBills.Remove(utilityBill);
                    context.SaveChanges();
                    Console.WriteLine("Utility Bill deleted.");
                }
                // Delete a Product Purchase
                var productPurchase =
context.ProductPurchases.FirstOrDefault(p => p.InvoiceNumber == "PP001");
                if (productPurchase != null)
                {
                    context.ProductPurchases.Remove(productPurchase);
                    context.SaveChanges();
                    Console.WriteLine("Product Purchase deleted.");
                }
                // Delete a Subscription Service
                var subscription =
context.SubscriptionServices.FirstOrDefault(s => s.InvoiceNumber == "SS001");
                if (subscription != null)
                {
                    context.SubscriptionServices.Remove(subscription);
```

```

        context.SaveChanges();
        Console.WriteLine("Subscription Service deleted.");
    }
}
}
}
}
}
}
}

```

Output:

```

Utility Bill deleted.
Product Purchase deleted.
Subscription Service deleted.

```

Advantages of TPC in Entity Framework Core

- **Performance Benefits:** Since all properties are in one table per concrete class, queries do not require joins, which can improve performance.
- **Simplicity:** The database schema is straightforward, with one table per concrete class.
- **No Null Columns:** Unlike TPH, there are no null columns because each table contains only relevant properties.
- **Easy Maintenance:** Separate tables make it easier to manage indexes and constraints specific to each entity type.

Drawbacks of TPC in Entity Framework Core

- **Data Redundancy:** Common properties inherited from the base class are repeated in each derived class's table, leading to redundancy.
- **Schema Changes:** Adding a new property to the base class requires updating all derived tables.
- **Limited Support for Polymorphic Queries:** Queries that need to retrieve data across the entire inheritance hierarchy may become more complex as they need to union results from multiple tables.

Table Per Concrete Type (TPC) inheritance in Entity Framework Core provides an efficient way to map inheritance hierarchies in EF Core by eliminating the need for joins when querying derived types. While it offers performance benefits and simplicity, it comes with the trade-off of data redundancy and potential maintenance overhead.

In the next article, I will discuss [Transactions in Entity Framework Core](#) with Examples. In this article, I explain Table Per Concrete Type (TPC) Inheritance in Entity Framework Core with Examples. I hope you enjoy this TPC Inheritance in EF Core article.